React Native Study Notes

Project: Warhammer40KApp

Goal: explain the React Native and React concepts used in the app: components, JSX, state, effects, touchables, lists, modals, props, and navigation params.

Table of Contents

1. What React Native Is

React Native is a framework for building mobile apps with React and JavaScript or TypeScript. Instead of writing Android UI in Kotlin/Java and iOS UI in Swift, you write React components. React Native converts those components into native mobile UI.

In a web React app, you use elements like:

```
<div>
  <p>Hello</p>
</div>
```

In React Native, you use mobile components like:

```
<View>
  <Text>Hello</Text>
</View>
```

Common React Native components used in this project

- `View`: a layout container, similar to a `div` on the web.
- `Text`: displays text. In React Native, text must usually be inside `Text`.
- `TextInput`: lets the user type input.
- `Pressable`: handles presses/taps.
- `FlatList`: renders scrollable lists efficiently.
- `ImageBackground`: displays an image as a background with content on top.
- `Modal`: displays content above the current screen.
- `KeyboardAvoidingView`: moves content so the keyboard does not cover inputs.
- `TouchableWithoutFeedback`: detects a touch without visual feedback, used here to dismiss the keyboard.

> **Defense answer:** React Native lets the app be built with React components while still rendering native mobile UI. The project uses React Native components such as `View`, `Text`, `Pressable`, `FlatList`, and `Modal` to build mobile screens.

2. Components, JSX, and Rendering

What a component is

A component is a reusable piece of UI written as a function. It returns JSX, which looks like HTML but is actually JavaScript/TypeScript syntax used by React.

```
const AddUnitButton = ({ onPress }: AddUnitButtonProps) => {
  return (
    <Pressable onPress={onPress}>
      <Text>Add Unit</Text>
    </Pressable>
  );
};
```

This component receives an `onPress` prop and renders a button-like UI.

Rendering

Rendering means React runs the component function and creates the UI from the returned JSX.

A component re-renders when:

- its state changes
- its props change
- context or Redux data it reads changes

React Native styles

React Native styling uses JavaScript objects instead of CSS files. In this project, styles are stored in separate files under `src/styles`.

```
<Pressable
  style={[styles.button, { backgroundColor: colors.button }]}
  onPress={onPress}
>
```

The style prop receives an array. React Native combines the objects from left to right. This lets the app combine fixed styles with dynamic theme colors.

> **Defense answer:** Screens and reusable UI pieces are written as components. Components return JSX, and React re-renders them when their data changes.

3. useState

What useState does

`useState` stores data that can change while the component is being used. When state changes, React re-renders the component with the new value.

How to write useState

```
const [value, setValue] = useState(initialValue);
```

- `value` is the current state value.
- `setValue` is the function used to change it.
- `initialValue` is the starting value.

Example: splash screen state

In `App.tsx`:

```
const [showSplashScreen, setShowSplashScreen] = useState(true);
```

This means the splash screen starts visible. Later, the app calls:

```
setShowSplashScreen(false);
```

That changes the state. React re-renders `AppContent`, and the app can stop showing the splash screen.

Example: modal state

In `ArmyCompositionScreen`:

```
const [modalVisible, setModalVisible] = useState(false);
```

The modal starts hidden because the state is `false`.

```
<AddUnitButton onPress={() => setModalVisible(true)} />
```

When the user presses the button, the state changes to `true`, so the modal becomes visible.

```
<SelectUnitModal
  visible={modalVisible}
  onClose={() => setModalVisible(false)}
/>
```

When the modal closes, the state changes back to `false`.

Example: typed state

In `AuthContext`:

```
const [currentUser, setCurrentUser] = useState<User | null>(null);
```

The TypeScript type `User | null` means `currentUser` can be a Firebase user or `null`. It starts as `null` because the app does not know yet whether someone is logged in.

> **Defense answer:** I use `useState` for values that change while the app is running, such as whether a modal is visible, whether the splash screen is showing, form errors, selected units, selected army, and the current Firebase user.

4. useEffect

What useEffect does

`useEffect` runs side effects. A side effect is work that happens outside normal rendering, such as loading data, starting a Firebase listener, reading AsyncStorage, or setting a timer.

How to write useEffect

```
useEffect(() => {
  // side effect code here
}, [dependencies]);
```

The dependency array controls when the effect runs.

- `[]` : run once when the component mounts.
- `[currentUser]` : run when `currentUser` changes.
- No dependency array: run after every render. This is usually not wanted.

Example: splash timer

In `App.tsx` :

```
useEffect(() => {
  const timer = setTimeout(() => {
    setShowSplashScreen(false);
  }, 2000);

  return () => clearTimeout(timer);
}, []);
```

This effect runs once. It starts a timer. After two seconds, the timer hides the splash screen.

The return function is cleanup:

```
return () => clearTimeout(timer);
```

If the component unmounts before the timer finishes, React clears the timer.

Example: Firebase auth listener

In `AuthContext` :

```
useEffect(() => {
  const unsubscribe = onAuthStateChanged(auth, (user) => {
    setCurrentUser(user);
    setLoading(false);
```

```
  });

  return unsubscribe;
}, []);
```

This effect starts a Firebase listener once. Firebase calls the callback when the auth state changes. The returned `unsubscribe` function is cleanup, so React can stop the listener later.

Example: Firestore listener with dependencies

In `useLoadArmyCompositions`:

```
useEffect(() => {
  if (!currentUser) {
    return;
  }

  const unsubscribe = subscribeToArmyCompositions(
    database,
    currentUser.uid,
    (armyCompositions) => {
      dispatch(armyCompositionsLoaded(armyCompositions));
    },
  );

  return unsubscribe;
}, [currentUser, dispatch]);
```

This effect depends on `currentUser`. When a user logs in, the effect subscribes to that user's Firestore army compositions. When the user changes or the component unmounts, React calls the cleanup function.

> **Defense answer:** I use `useEffect` for side effects such as timers, loading saved theme data, loading armies from Firestore, and subscribing to Firebase Auth or Firestore updates. Cleanup functions prevent timers or listeners from staying active forever.

5. Pressable, TouchableOpacity, and TouchableWithoutFeedback

Pressable

`Pressable` is a React Native component for handling press interactions. It is more flexible than older touchable components because it can react to pressed, hovered, focused, and disabled states.

How to write Pressable

```
<Pressable onPress={handlePress}>
  <Text>Press me</Text>
</Pressable>
```

Example from your button components

```
const AddUnitButton = ({ onPress }: AddUnitButtonProps) => {
  return (
    <Pressable
      style={[styles.button, { backgroundColor: colors.button }]}
      onPress={onPress}
    >
      <Text style={[styles.buttonText, { color: colors.buttonText }]}>
        Add Unit
      </Text>
    </Pressable>
  );
};
```

The button does not decide what should happen when it is pressed. It receives `onPress` as a prop from its parent. That makes the component reusable.

Pressable with pressed state

In `ArmyCompositionCard`, the trash icon changes color when pressed:

```
<Pressable style={styles.deleteButton} onPress={onDelete}>
  {({ pressed }) => (
    <Ionicons
      name="trash-outline"
      size={24}
      color={pressed ? "#ff4d4d" : colors.text}
    />
  )}
</Pressable>
```

Here, `Pressable` gives a `pressed` value. When the user is pressing, `pressed` is `true`. The icon color changes based on that value.

TouchableOpacity

`TouchableOpacity` is an older React Native touch component. When pressed, it automatically lowers the opacity of its children, creating a fade effect.

How to write TouchableOpacity

```
<TouchableOpacity onPress={handlePress} activeOpacity={0.7}>
  <Text>Press me</Text>
</TouchableOpacity>
```

`activeOpacity` controls how transparent the component becomes while pressed. A value like `0.7` means the component becomes 70% opaque.

> **Project note:** Your project mostly uses `Pressable`, not `TouchableOpacity`. If asked, you can explain that `Pressable` is the more flexible modern option, while `TouchableOpacity` is useful when you only want a simple opacity feedback effect.

TouchableWithoutFeedback

`TouchableWithoutFeedback` detects a touch without showing visual feedback. Your project uses it to dismiss the keyboard when the user taps outside a form.

```
<TouchableWithoutFeedback onPress={Keyboard.dismiss}>
  <KeyboardAvoidingView>
    ...
  </KeyboardAvoidingView>
</TouchableWithoutFeedback>
```

When the user taps outside the input, `Keyboard.dismiss` runs and hides the keyboard.

> **Defense answer:** The app uses `Pressable` for buttons and interactive cards because it handles tap interactions and can react to pressed state. `TouchableWithoutFeedback` is used around forms to dismiss the keyboard without adding a visual button effect.

6. FlatList, renderItem, and keyExtractor

What FlatList does

`FlatList` renders scrollable lists efficiently. Instead of rendering every item at once, it renders what is needed for the visible area. This is better for performance when lists become large.

Basic FlatList shape

```
<FlatList
  data={items}
  keyExtractor={(item) => item.id}
  renderItem={({ item }) => (
    <Text>{item.name}</Text>
  )}
/>
```

- `data` : the array to display.
- `keyExtractor` : gives each item a stable unique key.
- `renderItem` : tells FlatList how to render one item.

renderItem

`renderItem` receives an object. The most important property is `item` .

```
const renderArmyCompositionItem = ({ item }: { item: ArmyComposition }) => {
  return (
    <ArmyCompositionCard
      armyComposition={item}
      onPress={() =>
        navigation.navigate("ArmyComposition", {
          armyComposition: item,
        })
      }
      onDelete={() => deleteComposition(item.id)}
    />
  );
};
```

Here, one `item` is one `ArmyComposition` . The function returns one `ArmyCompositionCard` .

Using renderItem in FlatList

```
<FlatList
  data={armyCompositions}
  keyExtractor={(item) => item.id.toString()}
  renderItem={renderArmyCompositionItem}
```

```
  contentContainerStyle={styles.list}
/>
```

This means: for every army composition in `armyCompositions`, call `renderArmyCompositionItem` and display the returned card.

keyExtractor

`keyExtractor` gives React a unique key for each row. React uses keys to track which list items changed, were added, or were removed.

```
keyExtractor={(item) => item.id.toString()}
```

The key should be stable and unique. In your app, using the database or UUID id is good because each army composition has its own id.

> **Important:** Avoid using the array index as the key if the list can change order or items can be added/deleted. Index keys can make React confuse one row with another.

Example: selected units FlatList

```
<FlatList
  style={styles.unitList}
  data={selectedUnits}
  contentContainerStyle={styles.unitListContent}
  keyExtractor={(item) => item.armyCompositionUnitId.toString()}
  renderItem={renderSelectedUnitItem}
/>
```

This uses `armyCompositionUnitId`, not just `unit.id`, because the same unit type can be added multiple times. Each selected copy needs its own unique key.

Example: army selection FlatList

```
<FlatList
  data={armies}
  keyExtractor={(item) => item.id.toString()}
  renderItem={renderArmyItem}
/>
```

This renders the selectable faction cards when creating an army composition.

> **Defense answer:** I use `FlatList` for lists because it is optimized for mobile scrolling. `renderItem` defines how each row looks, and `keyExtractor` gives React a stable id so it can update the list correctly.

7. Modal

What Modal does

A `Modal` displays content above the current screen. It is useful for temporary workflows where the user should not fully navigate away.

Example from SelectUnitModal

```
<Modal visible={visible} animationType="slide">
  <SafeAreaView>
    <Pressable onPress={onClose}>
      <Text>Close</Text>
    </Pressable>

    <FlatList
      data={units}
      renderItem={renderUnitItem}
      keyExtractor={(item) => item.id.toString()}
    />
  </SafeAreaView>
</Modal>
```

`visible` controls whether the modal is shown. `animationType="slide"` makes it slide into view.

How this connects to state

In `ArmyCompositionScreen`:

```
const [modalVisible, setModalVisible] = useState(false);
```

The modal receives that state as a prop:

```
<SelectUnitModal
  visible={modalVisible}
  onClose={() => setModalVisible(false)}
  onAddUnit={addUnit}
/>
```

Pressing the add unit button changes the state to `true`. Pressing close changes it back to `false`.

> **Defense answer:** The unit picker is a modal because the user is temporarily choosing a unit while staying in the army composition screen. The parent screen controls visibility with state.

8. Props

What props are

Props are values passed from a parent component to a child component. They are how components receive data and callbacks.

Props flow downward:

```
Parent component
  -> passes props
  -> child component receives props
  -> child component uses props to render or call callbacks
```

Props example: AddUnitButton

```
type AddUnitButtonProps = {
  onPress: () => void;
};
```

This type says the component needs one prop named `onPress`. That prop must be a function that returns nothing.

```
const AddUnitButton = ({ onPress }: AddUnitButtonProps) => {
  return (
    <Pressable onPress={onPress}>
      <Text>Add Unit</Text>
    </Pressable>
  );
};
```

The parent decides what `onPress` does:

```
<AddUnitButton onPress={() => setModalVisible(true)} />
```

So the child button is reusable. It only knows that it should call `onPress` when pressed.

Props example: UnitCard

```
type UnitCardProps = {
  unit: Unit;
  buttonText: string;
  onPress: () => void;
};
```

This component receives:

- `unit`: the unit data to display.
- `buttonText`: the text on the button, such as `+` or `-`.
- `onPress`: what happens when the button is pressed.

The same `UnitCard` can be reused for adding and removing units because behavior comes from props.

Props example: Header

```
type HeaderProps = {
  title: string;
  canGoBack?: boolean;
  onBackPress?: () => void;
};
```

The question mark means optional. `canGoBack` and `onBackPress` do not have to be passed every time.

```
const Header = ({ title, canGoBack = false, onBackPress }: HeaderProps) => {
  ...
};
```

`canGoBack = false` gives a default value. If the parent does not pass `canGoBack`, it will be `false`.

> **Defense answer:** Props make components reusable. A child component receives data and callback functions from its parent, but the parent stays in control of what should happen.

9. Navigation Params

What navigation params are

Navigation params are data passed from one screen to another through React Navigation.

Props are passed from parent components to child components. Params are passed between screens through navigation.

Typed params in BattleForgeStack

```
export type BattleForgeStackParamList = {
  Home: undefined;
  CreateArmyComposition: undefined;
  ArmyComposition: { armyComposition: ArmyComposition };
};
```

This type says:

- `Home` does not need params.
- `CreateArmyComposition` does not need params.
- `ArmyComposition` needs one param named `armyComposition`.

Passing params

In `HomeScreen`, when the user presses an army composition card:

```
navigation.navigate("ArmyComposition", {
  armyComposition: item,
});
```

This opens the `ArmyComposition` screen and passes the selected army composition as route data.

Receiving params

In `ArmyCompositionScreen`:

```
type ArmyCompositionRouteProp = RouteProp<
  BattleForgeStackParamList,
  "ArmyComposition"
>;
```

This creates a TypeScript type for the route of the `ArmyComposition` screen.

```
const route = useRoute<ArmyCompositionRouteProp>();
```

```
const { armyComposition } = route.params;
```

`useRoute` gives access to the current screen's route object. `route.params` contains the data that was passed during navigation.

Props versus params

- **Props:** passed from parent component to child component.
- **Params:** passed from one screen to another through navigation.

Example difference

```
<ArmyCompositionCard
  armyComposition={item}
  onPress={() => navigation.navigate("ArmyComposition", {
    armyComposition: item,
  })}
/>
```

Here, `armyComposition={item}` is a prop passed to `ArmyCompositionCard`.

Inside `navigation.navigate`, `armyComposition: item` is a navigation param passed to the next screen.

> **Defense answer:** I use props to pass data and functions into reusable components. I use navigation params to pass data between screens, for example sending the selected army composition from Home to the detail screen.

10. Wrapping and Providers

What wrapping means

Wrapping means placing one component around another component.

```
<Parent>
  <Child />
</Parent>
```

Here, `Parent` wraps `Child`. In React, this often means the parent provides layout, behavior, or shared data to everything inside it.

Wrapping is not always visual. A wrapper can change layout, but a provider wrapper usually gives access to data or app services.

The children prop

When a component wraps other JSX, that inner JSX is received as `children`.

```
type AuthProviderProps = {
  children: ReactNode;
};
```

```
export const AuthProvider = ({ children }: AuthProviderProps) => {
  return (
    <AuthContext.Provider value={{ currentUser, loading }}>
      {children}
    </AuthContext.Provider>
  );
};
```

The provider does not need to know exactly what its children are. It simply renders them inside the provider.

Provider concept

A provider is a wrapper component that makes something available to all components inside it.

```
Provider
  -> supplies value/service/state
  -> children inside provider can use it
```

For example, `AuthProvider` supplies authentication state. Components inside it can call `useAuth()`.

Your App.tsx provider tree

In `App.tsx`, the app is wrapped like this:

```
<Provider store={store}>
  <PersistGate loading={null} persistor={persistor}>
    <SafeAreaProvider>
      <AuthProvider>
        <ThemeProvider>
          <AppContent />
        </ThemeProvider>
      </AuthProvider>
    </SafeAreaProvider>
  </PersistGate>
</Provider>
```

This means `AppContent` is inside every provider. Because it is inside them, it can use Redux, persisted state, safe-area behavior, authentication context, and theme context.

Redux Provider

```
<Provider store={store}>
```

This `Provider` comes from `react-redux`. It makes the Redux store available to all child components.

Without this wrapper, hooks like these would not work:

```
const dispatch = useAppDispatch();
const armyCompositions = useAppSelector(selectArmyCompositions);
```

> **Defense answer:** The Redux provider wraps the app so all screens and hooks can access the central Redux store.

PersistGate

```
<PersistGate loading={null} persistor={persistor}>
```

`PersistGate` comes from Redux Persist. It waits while persisted Redux state is loaded from AsyncStorage.

This matters because Redux Persist may have saved army composition state locally. `PersistGate` prevents the app UI from rendering before that state has been rehydrated.

> **Defense answer:** PersistGate delays rendering until persisted Redux data has been restored from local storage.

SafeAreaProvider

```
<SafeAreaProvider>
```

`SafeAreaProvider` gives the app information about safe screen areas. This helps content avoid notches, status bars, rounded corners, and bottom gesture areas.

Screens can then use `SafeAreaView`:

```
<SafeAreaView style={styles.container}>
  ...
</SafeAreaView>
```

> **Defense answer:** SafeAreaProvider makes safe-area information available so screens can avoid device-specific blocked areas.

AuthProvider

```
<AuthProvider>
```

`AuthProvider` is your own provider. It listens to Firebase Authentication and supplies:

- `currentUser`
- `loading`

Because `AppContent` is inside `AuthProvider`, it can do this:

```
const { currentUser, loading } = useAuth();
```

That lets `AppContent` decide whether to show the logged-in app or the authentication screens.

ThemeProvider

```
<ThemeProvider>
```

`ThemeProvider` is also your own provider. It supplies:

- `theme`
- `toggleTheme`

Because screens and components are inside `ThemeProvider`, they can call:

```
const { theme, toggleTheme } = useTheme();
```

The drawer navigation also uses the theme, so the drawer needs to be rendered inside `ThemeProvider`.

NavigationContainer

`NavigationContainer` is inside `AppContent`:

```
return (
  <NavigationContainer>
    {currentUser ? <BattleForgeInformationDrawerStack /> : <AuthStack />}
  </NavigationContainer>
);
```

`NavigationContainer` manages navigation state. It must wrap the navigators, such as `AuthStack` and `BattleForgeInformationDrawerStack`.

It is rendered after the loading check, so the app does not show navigation until authentication, fonts, and the splash delay are ready.

Why order matters

Provider order matters when an inner component needs access to something from an outer provider.

For example, `AppContent` calls `useAuth()`, so `AppContent` must be inside `AuthProvider`.

```
<AuthProvider>
  <AppContent />
</AuthProvider>
```

If `AppContent` were outside `AuthProvider`, it would not receive the real auth context value.

Redux has the same idea. Components that use Redux hooks must be inside the Redux `Provider`.

Provider scope

A provider only affects components inside it.

```
Provider A
  -> Component inside can use Provider A

Component outside
  -> cannot use Provider A's value
```

This is called provider scope. In your app, the providers wrap almost everything because the app needs Redux, auth, theme, and safe-area support globally.

Wrapper components versus provider components

- **Wrapper component:** any component that surrounds children. It may provide layout, styling, behavior, or data.
- **Provider component:** a specific kind of wrapper that supplies shared data or services to its children.

For example, `KeyboardAvoidingView` is a wrapper for layout behavior around forms. `AuthProvider` is a provider for authentication data.

> **Defense summary:** The app uses wrapping to give child components access to global tools and state. The Redux Provider supplies the store, PersistGate restores persisted Redux state, SafeAreaProvider handles device safe areas, AuthProvider supplies Firebase auth state, ThemeProvider supplies theme state, and NavigationContainer manages screen navigation.

11. Context and AsyncStorage

What Context is

React Context is a way to share data with many components without manually passing props through every level of the component tree.

Normally, data passed through props works like this:

```
App
  -> Screen
  -> Form
  -> Button
```

If every level needs to pass the same value just so a deeply nested component can use it, the code becomes messy. Context solves this by letting a provider put a value into a shared place, and any child component inside that provider can read it.

```
Provider gives value
  -> any child inside provider can read value with useContext
```

Where Context is used in this project

Your project uses Context for two app-wide concerns:

- `AuthContext`: stores the current Firebase user and loading state.
- `ThemeContext`: stores the selected light/dark theme and the function to toggle it.

Both are good uses of Context because many screens need the same data.

The Context pattern

A typical Context setup has four parts:

- A type describing the context value.
- A context object created with `createContext`.
- A provider component that owns the real state.
- A custom hook that makes reading the context easier.

AuthContext type

In `src/contexts/AuthContext.tsx`:

```
type AuthContextType = {
  currentUser: User | null;
  loading: boolean;
};
```

This says the authentication context provides two values:

- `currentUser`: either a Firebase `User` object or `null`.
- `loading`: a boolean that says whether Firebase is still checking the saved login session.

Creating the context

```
export const AuthContext = createContext<AuthContextType>({
  currentUser: null,
  loading: true,
});
```

`createContext` creates the shared context object. The value passed into `createContext` is the default value. Components normally receive the real value from the provider, but the default value gives TypeScript and React a safe fallback.

The provider component

```
export const AuthProvider = ({ children }: AuthProviderProps) => {
  const [currentUser, setCurrentUser] = useState<User | null>(null);
  const [loading, setLoading] = useState(true);

  useEffect(() => {
    const unsubscribe = onAuthStateChanged(auth, (user) => {
      setCurrentUser(user);
      setLoading(false);
    });

    return unsubscribe;
  }, []);

  return (
    <AuthContext.Provider value={{ currentUser, loading }}>
      {children}
    </AuthContext.Provider>
  );
};
```

The provider owns the actual state. It watches Firebase Auth and puts the current values into:

```
value={{ currentUser, loading }}
```

Everything inside `AuthContext.Provider` can read those values.

What children means

```
type AuthProviderProps = {
  children: ReactNode;
```

```
};
```

`children` means the content placed inside the provider component.

```
<AuthProvider>
  <ThemeProvider>
    <AppContent />
  </ThemeProvider>
</AuthProvider>
```

Here, `ThemeProvider` and `AppContent` are inside `AuthProvider`, so they can access the auth context.

Reading context with useContext

The hook `useAuth` is in `src/hooks/useAuth.ts`:

```
export const useAuth = () => {
  return useContext(AuthContext);
};
```

This is a custom hook. Instead of writing this everywhere:

```
const authContext = useContext(AuthContext);
```

The app can write:

```
const { currentUser, loading } = useAuth();
```

This makes the code shorter and keeps context access consistent.

ThemeContext type

In `src/contexts/ThemeContext.tsx`:

```
type Theme = "light" | "dark";

type ThemeContextType = {
  theme: Theme;
  toggleTheme: () => void;
};
```

`Theme` is a union type. It only allows `"light"` or `"dark"`. This prevents invalid theme values.

Default context value

```
export const ThemeContext = createContext<ThemeContextType>({
  theme: "dark",
  toggleTheme: () => {},
});
```

The default `toggleTheme` is an empty function. It is only a fallback before the real provider value exists. The real `toggleTheme` comes from `ThemeProvider`.

ThemeProvider

```
export const ThemeProvider = ({ children }: ThemeProviderProps) => {
  const [theme, setTheme] = useState<Theme>("dark");

  useEffect(() => {
    const loadTheme = async () => {
      const savedTheme = await AsyncStorage.getItem("theme");

      if (savedTheme === "light" || savedTheme === "dark") {
        setTheme(savedTheme);
      }
    };

    loadTheme();
  }, []);

  const toggleTheme = async () => {
    const newTheme = theme === "dark" ? "light" : "dark";

    setTheme(newTheme);

    await AsyncStorage.setItem("theme", newTheme);
  };

  return (
    <ThemeContext.Provider value={{ theme, toggleTheme }}>
      {children}
    </ThemeContext.Provider>
  );
};
```

This provider stores the current theme in React state, loads the saved theme when the app starts, and saves the new theme when the user toggles it.

How Context differs from Redux

Context and Redux can both share data, but they are used for different types of state in this app.

- `AuthContext` and `ThemeContext` handle small global values.
- Redux handles the army composition feature state, which has more actions such as load, add, update, delete, and clear.

> **Defense answer:** The project uses Context for small global app values like authentication and theme. Redux is used for army compositions because that feature has more complex state changes.

What AsyncStorage is

`AsyncStorage` is local persistent storage on the device. It works like a small key-value storage system for the app.

It is called async because reading and writing storage takes time. The methods return promises, so the code uses `async` and `await`.

AsyncStorage stores strings

AsyncStorage stores string values. In this app, the theme is already a string, so it can be saved directly:

```
await AsyncStorage.setItem("theme", newTheme);
```

For objects or arrays, you normally convert them to JSON first:

```
await AsyncStorage.setItem("userSettings", JSON.stringify(settings));
```

And read them back with:

```
const savedSettings = await AsyncStorage.getItem("userSettings");

if (savedSettings) {
  const settings = JSON.parse(savedSettings);
}
```

Reading from AsyncStorage

In `ThemeContext`:

```
const savedTheme = await AsyncStorage.getItem("theme");
```

`getItem` reads a value by key. Here, the key is `"theme"`. If no value exists, it returns `null`.

```
if (savedTheme === "light" || savedTheme === "dark") {
  setTheme(savedTheme);
}
```

This check is important because AsyncStorage can technically contain any string. The app only accepts the two valid theme values.

Why the async function is inside useEffect

The effect is written like this:

```
useEffect(() => {
  const loadTheme = async () => {
    const savedTheme = await AsyncStorage.getItem("theme");
    ...
  };

  loadTheme();
}, []);
```

The `useEffect` callback itself should not be marked `async`, because an async function returns a promise. React expects the effect to return either nothing or a cleanup function. So the common pattern is to define an async function inside the effect and then call it.

Writing to AsyncStorage

```
const toggleTheme = async () => {
  const newTheme = theme === "dark" ? "light" : "dark";

  setTheme(newTheme);

  await AsyncStorage.setItem("theme", newTheme);
};
```

This updates the UI immediately with `setTheme(newTheme)`, then saves the choice for the next app start with `AsyncStorage.setItem`.

AsyncStorage and Redux Persist

AsyncStorage is also used indirectly in `src/store/index.ts`:

```
const persistConfig = {
  key: "root",
  storage: AsyncStorage,
};
```

This tells Redux Persist to save Redux state into AsyncStorage. That is how persisted Redux data can survive app restarts.

AsyncStorage versus Firestore

- **AsyncStorage:** local device storage, good for small app preferences or cached data.
- **Firestore:** remote cloud database, good for user data that should be saved outside the device and synchronized.

The theme is stored in AsyncStorage because it is a local preference. Army compositions are stored in Firestore because they are user data connected to an account.

> **Defense answer:** AsyncStorage is used for local persistence. In this app, it saves the theme preference directly and is also used by Redux Persist to save Redux state locally. Firestore is used for remote user data.

12. Short Defense Answers

What is React Native?

React Native is a framework that lets us build mobile apps with React components. The components render to native mobile UI instead of browser HTML.

What is useState?

`useState` stores a value that can change. When the setter function changes the value, React re-renders the component.

What is useEffect?

`useEffect` runs side effects such as timers, subscriptions, loading data, and reading storage. Its dependency array controls when it runs, and its return function is cleanup.

Why use Pressable?

`Pressable` handles touch interactions and can respond to pressed state. The project uses it for buttons, cards, and icons.

What is TouchableOpacity?

`TouchableOpacity` is a touch component that automatically lowers opacity when pressed. It is useful for simple visual press feedback, although this project mostly uses `Pressable`.

Why use TouchableWithoutFeedback?

The project uses it to detect taps outside forms and call `Keyboard.dismiss`, hiding the keyboard without showing visual feedback.

Why use FlatList?

`FlatList` is optimized for scrollable lists. It uses `data`, `renderItem`, and `keyExtractor` to render list rows efficiently.

What is renderItem?

`renderItem` is a function that receives one list item and returns the component that should be shown for that row.

What is keyExtractor?

`keyExtractor` returns a stable unique key for each item. React uses keys to track list items correctly during updates.

What are props?

Props are values passed from a parent component to a child component. They can be normal data or callback functions.

What are navigation params?

Navigation params are data passed from one screen to another through React Navigation.

What does wrapping mean in React?

Wrapping means placing one component around another. A wrapper can provide layout, behavior, or shared data to its children.

What is a provider?

A provider is a wrapper component that makes a value or service available to all components inside it.

Why does provider order matter?

A component can only use a provider's value if it is rendered inside that provider. That is why `AppContent` is inside Redux, auth, theme, and safe-area providers.

What is Context?

Context shares data with all components inside a provider without passing props through every level. This app uses it for auth and theme.

What is a Provider?

A provider is the component that supplies the context value to its children. For example, `AuthProvider` supplies `currentUser` and `loading`.

Why use custom hooks like useAuth and useTheme?

They wrap `useContext` so components can read context in a clean and consistent way.

What is AsyncStorage?

AsyncStorage is local persistent key-value storage on the device. It stores strings and uses asynchronous methods such as `getItem` and `setItem`.

Why not make useEffect async directly?

An async function returns a promise, but React expects an effect to return nothing or a cleanup function. That is why the async function is defined inside the effect and then called.